# Assignment - Module 1

Meenu S

January 1, 2026

## Introduction

Linear algebra forms the foundational language of modern mathematics and its applications, particularly through the study of vectors, matrices, and linear transformations. Matrix theory provides a unified framework to understand these concepts by linking algebraic structures with geometric intuition and computational methods.

The concepts of change of basis and coordinate representation highlight how the same vector can be interpreted differently under various reference frames, emphasizing the role of matrices in linear transformations. Vector length, norms, and unit vectors introduce quantitative measures of magnitude and direction, while dot products and inner products establish a precise notion of orthogonality and angle between vectors.

Additionally, the Wronskian is studied as a determinant-based criterion to examine linear independence, illustrating how matrix methods extend naturally to the analysis of functions. Alongside theoretical discussions, this assignment incorporates Python-based visualizations using `matplotlib` to provide geometric insight and computational verification of results. These visual and computational approaches strengthen conceptual understanding by connecting abstract theory with concrete examples.

## 1 Change of Basis and Coordinate Matrices

Let $V$ be a finite-dimensional vector space over a field $\mathbb{F}$ and let $B = \{b_1, b_2, \ldots, b_n\}$ be an ordered basis of $V$.

Every vector $v \in V$ can be written uniquely as

$$v = c_1 b_1 + c_2 b_2 + \cdots + c_n b_n,$$

where $c_i \in \mathbb{F}$. The coordinate vector of $v$ with respect to the basis $B$ is

$$[v]_B = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix}.$$

## Transition Matrix Between Bases

Let $B = \{b_1, b_2, \ldots, b_n\}$ and $C = \{c_1, c_2, \ldots, c_n\}$ be two ordered bases of a vector space $V$.

Define the basis matrices

$$[B] = \begin{pmatrix} b_1 & b_2 & \cdots & b_n \end{pmatrix}, \qquad [C] = \begin{pmatrix} c_1 & c_2 & \cdots & c_n \end{pmatrix}.$$

(The basis elements are column vectors)

For any vector $v \in V$, we have

$$v = [B]\,[v]_B \quad \text{and} \quad v = [C]\,[v]_C.$$

Equating the two expressions,

$$[B]\,[v]_B = [C]\,[v]_C.$$

Multiplying both sides by $[C]^{-1}$, we obtain

$$[C]^{-1}[B]\,[v]_B = [v]_C.$$

Hence,

$$[v]_C = P_{C \leftarrow B}\,[v]_B,$$

where the transition matrix from basis $B$ to basis $C$ is

$$P_{C \leftarrow B} = [C]^{-1}[B].$$

## Inverse Transition Matrix

The transition matrix from $C$ to $B$ is the inverse of $P_{C \leftarrow B}$:

$$P_{B \leftarrow C} = \left(P_{C \leftarrow B}\right)^{-1},$$

and hence

$$[v]_B = P_{B \leftarrow C}\,[v]_C.$$

$*$Let the new basis be

$$B = \{b_1, b_2\} = \{(\cos x, \sin x),\ (-\sin x, \cos x)\}$$

and let the vector in standard coordinates be

$$v = (1, 1).$$

$$P_B = \begin{bmatrix} b_1 & b_2 \end{bmatrix} = \begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}$$

By definition, the coordinate vector $[v]_B$ satisfies

$$v = P_B[v]_B \quad \Longrightarrow \quad [v]_B = P_B^{-1}v$$

Since $P_B$ is orthogonal (rotation matrix),

$$P_B^{-1} = P_B^\top = \begin{bmatrix} \cos x & \sin x \\ -\sin x & \cos x \end{bmatrix}$$

$$[v]_B = P_B^\top v = \begin{bmatrix} \cos x & \sin x \\ -\sin x & \cos x \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos x + \sin x \\ -\sin x + \cos x \end{bmatrix} = \begin{bmatrix} \cos x + \sin x \\ \cos x - \sin x \end{bmatrix}$$

$$\cos 45^\circ = \sin 45^\circ = \frac{1}{\sqrt{2}}$$

$$[v]_B = \begin{bmatrix} \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} - \frac{1}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} \sqrt{2} \\ 0 \end{bmatrix}$$

When (1,1) is rotated anti-clockwise by 45 degrees, it will intersect the x-axis at this point (length of the vector (1,1) is $\sqrt{2}$ - Rotation as change of basis)

# Python Code Listing

Listing 1: Python Program for Change of Basis and Transition Matrices

```
import numpy as np
import matplotlib.pyplot as plt
import os

def get_basis_from_user(dim, basis_name):
    """Prompts the user to enter basis vectors."""
    print(f"--- Enter the {dim} {basis_name} vectors ---")
    basis_vectors = []
    for i in range(dim):
        while True:
            try:
                b_str = input(f"Basis vector {i+1}: ").split()
                if len(b_str) != dim:
                    print(f"Error: Please enter {dim} components.")
                    continue
                basis_vectors.append([float(x) for x in b_str])
                break
            except ValueError:
                print("Invalid input. Please enter numbers only.")
    # Create matrix with basis vectors as columns
    matrix = np.array(basis_vectors).T
    # Check if the basis is valid (i.e., vectors are linearly
       independent)
```

```
    if abs(np.linalg.det(matrix)) < 1e-9: # Use a tolerance for
       floating point numbers
        print(f"\nError: The vectors provided for '{basis_name}'
           are not linearly independent and do not form a valid
           basis.")
        return None
    return matrix

def general_change_of_basis():
    """
    Performs a change of basis for a vector from one arbitrary
       basis to another.
    """
    try:
        # 1. Specify the dimension
        dim_str = input("Enter the dimension of the vector space (2
           or 3): ")
        dim = int(dim_str)
        if dim not in [2, 3]:
            print("Invalid input. This script can only plot for
               dimensions 2 and 3.")
            return

        # 2. Get the initial basis (B)
        B = get_basis_from_user(dim, "initial basis (B)")
        if B is None: return

        # 3. Get the vector's coordinates in the initial basis
        print("\nEnter the vector's coordinates relative to the
           initial basis B, separated by spaces:")
        v_coords_B_str = input("Vector coordinates: ").split()
        v_coords_B = np.array([float(x) for x in
           v_coords_B_str]).reshape(-1, 1) # Column vector

        # 4. Get the new basis (B')
        B_prime = get_basis_from_user(dim, "new basis (B')")
        if B_prime is None: return

        # 5. Define the transition matrix C = (B')^-1 * B
        B_prime_inv = np.linalg.inv(B_prime)
        C = B_prime_inv @ B

        # 6. Calculate the new coordinates: v_coords_B' = C *
           v_coords_B
        v_coords_B_prime = C @ v_coords_B

        # 7. Print the results
```

```
np.set_printoptions(precision=3, suppress=True)
print("\n" + "="*40)
print("                RESULTS")
print("="*40)
print(f"Transition Matrix from B to B' (C):\n{C}\n")
print(f"Original coordinates in basis B:
   {v_coords_B.flatten()}")
print(f"New coordinates in basis B':
   {v_coords_B_prime.flatten()}")
print("="*40)

# 8. Generate plots for 2D or 3D cases
# The actual geometric vector in standard coordinates is B
   @ v_coords_B
v_geometric = (B @ v_coords_B).flatten()

print(f"\nGenerating {dim}D plots...")
fig = plt.figure(figsize=(16, 8))
save_path = 'general_basis_change.png'

if dim == 2:
    ax1 = fig.add_subplot(1, 2, 1)
    ax2 = fig.add_subplot(1, 2, 2)

    # --- Plot 1: Vector in Initial Basis B ---
    ax1.set_title("Vector in Initial Basis (B)")
    ax1.quiver(0, 0, B[0, 0], B[1, 0], angles='xy',
       scale_units='xy', scale=1, color='g', label=f'b1 =
       {B[:,0]}')
    ax1.quiver(0, 0, B[0, 1], B[1, 1], angles='xy',
       scale_units='xy', scale=1, color='b', label=f'b2 =
       {B[:,1]}')
    ax1.quiver(0, 0, v_geometric[0], v_geometric[1],
       angles='xy', scale_units='xy', scale=1, color='r',
       label=f'Coords = {v_coords_B.flatten()}')

    # --- Plot 2: Vector in New Basis B' ---
    ax2.set_title("Vector in New Basis (B')")
    ax2.quiver(0, 0, B_prime[0, 0], B_prime[1, 0],
       angles='xy', scale_units='xy', scale=1, color='g',
       label=f"b'1 = {B_prime[:,0]}")
    ax2.quiver(0, 0, B_prime[0, 1], B_prime[1, 1],
       angles='xy', scale_units='xy', scale=1, color='b',
       label=f"b'2 = {B_prime[:,1]}")
    ax2.quiver(0, 0, v_geometric[0], v_geometric[1],
       angles='xy', scale_units='xy', scale=1, color='r',
       label=f"Coords = {v_coords_B_prime.flatten()}")
```

```
        all_vectors = np.hstack([B, B_prime,
            v_geometric.reshape(-1,1)])
        max_val = np.max(np.abs(all_vectors)) + 1
        for ax in [ax1, ax2]:
            ax.set_aspect('equal', adjustable='box')
            ax.set_xlim(-max_val, max_val);
                ax.set_ylim(-max_val, max_val)
            ax.grid(); ax.legend()
            ax.axhline(0, color='k', lw=0.5); ax.axvline(0,
                color='k', lw=0.5)

    elif dim == 3:
        ax1 = fig.add_subplot(1, 2, 1, projection='3d')
        ax2 = fig.add_subplot(1, 2, 2, projection='3d')

        # --- Plot 1: Vector in Initial Basis B ---
        ax1.set_title("Vector in Initial Basis (B)")
        ax1.quiver(0, 0, 0, B[0,0], B[1,0], B[2,0], color='g',
            label=f'b1')
        ax1.quiver(0, 0, 0, B[0,1], B[1,1], B[2,1], color='b',
            label=f'b2')
        ax1.quiver(0, 0, 0, B[0,2], B[1,2], B[2,2], color='y',
            label=f'b3')
        ax1.quiver(0, 0, 0, v_geometric[0], v_geometric[1],
            v_geometric[2], color='r', label=f'Coords =
            {v_coords_B.flatten()}')

        # --- Plot 2: Vector in New Basis B' ---
        ax2.set_title("Vector in New Basis (B')")
        ax2.quiver(0, 0, 0, B_prime[0,0], B_prime[1,0],
            B_prime[2,0], color='g', label=f"b'1")
        ax2.quiver(0, 0, 0, B_prime[0,1], B_prime[1,1],
            B_prime[2,1], color='b', label=f"b'2")
        ax2.quiver(0, 0, 0, B_prime[0,2], B_prime[1,2],
            B_prime[2,2], color='y', label=f"b'3")
        ax2.quiver(0, 0, 0, v_geometric[0], v_geometric[1],
            v_geometric[2], color='r', label=f"Coords =
            {v_coords_B_prime.flatten()}")

        all_vectors = np.hstack([B, B_prime,
            v_geometric.reshape(-1,1)])
        max_val = np.max(np.abs(all_vectors)) + 1
        for ax in [ax1, ax2]:
            ax.set_xlim(-max_val, max_val);
                ax.set_ylim(-max_val, max_val);
                ax.set_zlim(-max_val, max_val)
```

```
                ax.set_xlabel('X'); ax.set_ylabel('Y');
                   ax.set_zlabel('Z')
                ax.legend()

        fig.tight_layout(pad=3.0)
        try:
            plt.savefig(save_path)
            if os.path.exists(save_path):
                print(f"Success! Plot saved as {save_path}.")
            else:
                print(f"Error: Plot file not created.")
        except Exception as e:
            print(f"--- FAILED TO SAVE PLOT: {e} ---")

    except ValueError:
        print("Invalid input. Please ensure you enter numbers
           only.")
    except np.linalg.LinAlgError as e:
        print(f"A matrix error occurred: {e}. One of the bases
           might be invalid.")
    except Exception as e:
        import traceback
        print(f"--- AN UNEXPECTED ERROR OCCURRED: {e} ---")
        print(f"TRACEBACK: {traceback.format_exc()}")

if __name__ == '__main__':
    # Activate the virtual environment first by running:
    # source .venv/bin/activate
    general_change_of_basis()
```

# OUTPUT

Enter the dimension of the vector space (2 or 3): 3
— Enter the 3 initial basis (B) vectors —
Basis vector 1: 1 1 1
Basis vector 2: 1 -1 1
Basis vector 3: 0 0 1

Enter the vector's coordinates relative to the initial basis B, separated by spaces:
Vector coordinates: 6 -1 -1

— Enter the 3 new basis (B') vectors —
Basis vector 1: 2 2 0
Basis vector 2: 0 1 1
Basis vector 3: 1 0 1

Transition Matrix from B to B' (C):

$$\begin{bmatrix} 0.25 & -0.25 & -0.25 \\ 0.5 & -0.5 & 0.5 \\ 0.5 & 1.5 & 0.5 \end{bmatrix}$$

Original coordinates in basis B: [ 6. -1. -1.]
New coordinates in basis B': [2. 3. 1.]

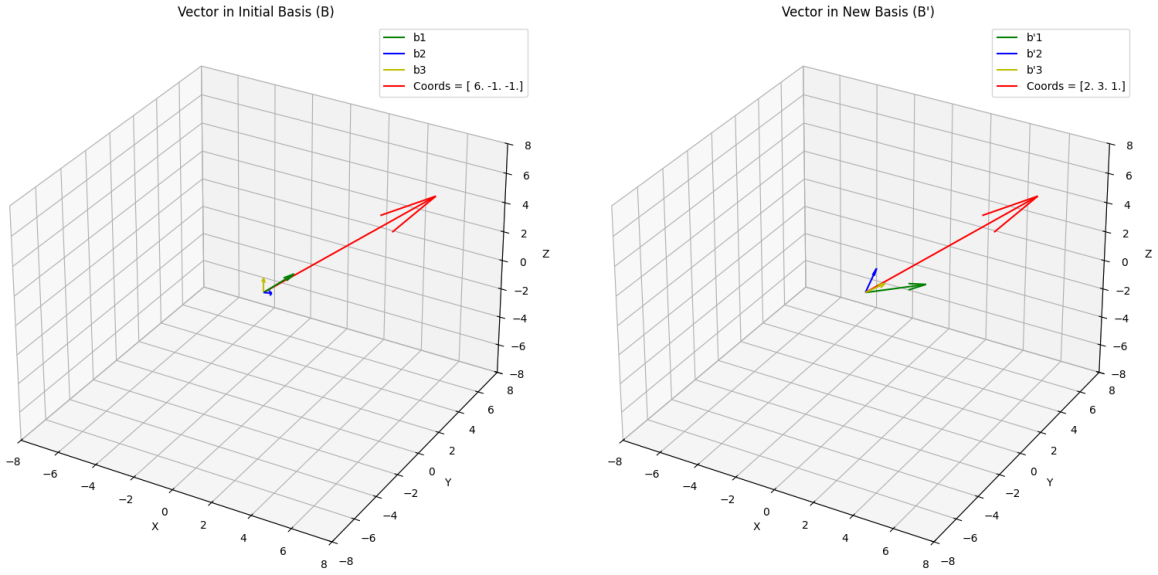



Figure 1: Representation of the vector in 2 different bases

# Explanation of the Code (Notes)

## Import Statements

- `import numpy as np`: Imports the NumPy package for numerical operations, especially matrix and vector calculations.
- `import matplotlib.pyplot as plt`: Imports Matplotlib for generating plots of vectors in the different bases.
- `import os`: Used to check if the plot file is successfully saved.

## The Function `get_basis_from_user`

- This function prompts the user to enter basis vectors.
- The matrix is created using these vectors as columns. It checks if the basis vectors are linearly independent by checking if the determinant of the matrix is non-zero.

- If the determinant is zero, the vectors are linearly dependent, and the function returns `None`.

### The Function `general_change_of_basis`

- This function handles the entire process of changing the basis, including obtaining the initial and new bases, calculating the transition matrix, and visualizing the result.

### Step-by-step Breakdown

- `dim = int(input("Enter the dimension of the vector space (2 or 3): "))`: Takes the dimension of the vector space as input (either 2D or 3D).
- `B = get_basis_from_user(dim, "initial basis (B)")`: Calls the `get_basis_from_user` function to obtain the initial basis vectors.
- `v_coords_B = np.array([float(x) for x in v_coords_B_str]).reshape(-1, 1)`: Converts the input vector coordinates into a column vector.
- `C = np.linalg.inv(B_prime) @ B`: Calculates the transition matrix $C$, where $C = (B')^{-1} \times B$.
- `v_coords_B_prime = C @ v_coords_B`: Calculates the vector's new coordinates in the new basis by multiplying the transition matrix with the original coordinates.
- The program then prints the transition matrix and the original and new coordinates.
- Finally, it generates 2D or 3D plots of the vectors in the initial and new bases using Matplotlib.

## 2 Vector Length and norms

Let

$$\mathbf{v} = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n.$$

The Euclidean norm (or magnitude) of $\mathbf{v}$ is defined as

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}.$$

This quantity represents the distance of the vector from the origin.

## Python Code

Listing 2: Python program to compute and visualize the Euclidean norm (length) of a vector

```
import numpy as np
import matplotlib.pyplot as plt
import traceback

def format_vector_for_display(v):
    """Formats a numpy array for display with commas, e.g.,
       [1.0000, 2.0000]."""
    return '[' + ', '.join(f'{x:.4f}' for x in v) + ']'

def get_dimension():
    """Gets a valid dimension from the user."""
    while True:
        try:
            dim_str = input("Enter the dimension of the vector
               space (e.g., 2 or 3): ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer for
               the dimension.")

def get_vector(dim):
    """Gets a valid vector from the user for a given dimension."""
    while True:
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements on one line, separated
           by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str = vector_str_input.split(',')

        if len(vector_str) != dim:
            print(f"Error: Incorrect number of elements. Please
               enter exactly {dim} elements.")
            continue

        try:
            v = np.array([float(x.strip()) for x in vector_str])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numbers.")

def calculate_and_plot_vector_length():
```

```
    """Calculates the length of a vector and plots it."""
    try:
        dim = get_dimension()
        v = get_vector(dim)

        length = np.linalg.norm(v)

        print(f"\nThe vector is: {format_vector_for_display(v)}")
        print(f"The length (magnitude) of the vector is:
           {length:.4f}")

        if dim == 2:
            fig, ax = plt.subplots(figsize=(8, 8))
            ax.set_title("2D Vector Plot")
            ax.quiver(0, 0, v[0], v[1], angles='xy',
               scale_units='xy',
                      scale=1, color='r',
                      label=f'Vector v =
                         {format_vector_for_display(v)}')
            ax.text(v[0] * 0.5, v[1] * 0.5,
                    f'Length = {length:.2f}', fontsize=12,
                       color='blue')

            ax.plot([v[0], v[0]], [0, v[1]], 'k--')
            ax.plot([0, v[0]], [v[1], v[1]], 'k--')

            max_val = np.max(np.abs(v)) * 1.5 if np.max(np.abs(v))
               > 0 else 1.0
            ax.set_xlim([-max_val, max_val])
            ax.set_ylim([-max_val, max_val])
            ax.set_xlabel("X-axis")
            ax.set_ylabel("Y-axis")
            ax.axhline(0, color='grey', lw=0.5)
            ax.axvline(0, color='grey', lw=0.5)
            ax.grid(True)
            ax.set_aspect('equal', adjustable='box')
            ax.legend()

            output_filename = 'vector_length_plot_2d.png'
            plt.savefig(output_filename)
            print(f"\n2D plot saved as '{output_filename}'")

        elif dim == 3:
            fig = plt.figure(figsize=(12, 12))
            fig.suptitle("3D Vector and Orthographic Projections",
               fontsize=16)
```

```
ax_3d = fig.add_subplot(2, 2, 1, projection='3d')
ax_3d.set_title("3D Perspective View")
ax_3d.quiver(0, 0, 0, v[0], v[1], v[2], color='r',
             label=f'v =
                {format_vector_for_display(v)}\nLength
                = {length:.2f}')

max_val = np.max(np.abs(v)) * 1.5 if np.max(np.abs(v))
   > 0 else 1.0
ax_3d.set_xlim([-max_val, max_val])
ax_3d.set_ylim([-max_val, max_val])
ax_3d.set_zlim([-max_val, max_val])
ax_3d.set_xlabel("X-axis")
ax_3d.set_ylabel("Y-axis")
ax_3d.set_zlabel("Z-axis")
ax_3d.legend()

ax_xy = fig.add_subplot(2, 2, 2)
ax_xy.set_title("X-Y Plane (Top View)")
ax_xy.quiver(0, 0, v[0], v[1], angles='xy',
             scale_units='xy', scale=1, color='b')
ax_xy.plot([v[0], v[0]], [0, v[1]], 'k--')
ax_xy.plot([0, v[0]], [v[1], v[1]], 'k--')
ax_xy.set_aspect('equal', adjustable='box')
ax_xy.grid(True)

ax_yz = fig.add_subplot(2, 2, 3)
ax_yz.set_title("Y-Z Plane (Side View)")
ax_yz.quiver(0, 0, v[1], v[2], angles='xy',
             scale_units='xy', scale=1, color='g')
ax_yz.plot([v[1], v[1]], [0, v[2]], 'k--')
ax_yz.plot([0, v[1]], [v[2], v[2]], 'k--')
ax_yz.set_aspect('equal', adjustable='box')
ax_yz.grid(True)

ax_xz = fig.add_subplot(2, 2, 4)
ax_xz.set_title("X-Z Plane (Front View)")
ax_xz.quiver(0, 0, v[0], v[2], angles='xy',
             scale_units='xy', scale=1, color='purple')
ax_xz.plot([v[0], v[0]], [0, v[2]], 'k--')
ax_xz.plot([0, v[0]], [v[2], v[2]], 'k--')
ax_xz.set_aspect('equal', adjustable='box')
ax_xz.grid(True)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
output_filename = 'vector_plot_3d_views.png'
plt.savefig(output_filename)
```

```
            print(f"\nMulti-view 3D plot saved as
                '{output_filename}'")

        else:
            print("\nPlotting is only supported for 2D and 3D
                vectors.")

    except KeyboardInterrupt:
        print("\nProgram interrupted by user.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        traceback.print_exc()

if __name__ == "__main__":
    calculate_and_plot_vector_length()
```

# OUTPUT

Enter the dimension of the vector space (e.g., 2 or 3): 3
Enter the 3 elements on one line, separated by commas (e.g., '1,2,3'): 2,0,6
The vector is: [2.0000, 0.0000, 6.0000]
The length (magnitude) of the vector is: 6.3246

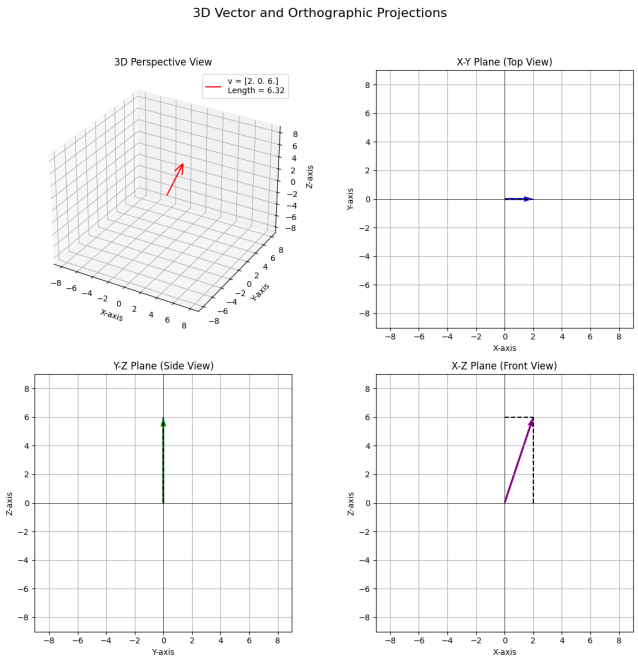



Figure 2: 3D vector showing its magnitude/length

Listing 3: Python program for computing norms of vectors and norm of their sum

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # Necessary for 3D plotting

def format_vector_for_display(v):
    """Formats a numpy array for display with commas to four
       decimal places."""
    return '[' + ', '.join(f'{x:.4f}' for x in v) + ']'

def get_dimension():
    """
    Asks the user to input the dimension of the vector space.
    """
    while True:
        try:
            dim = int(input("Enter the dimension of the vector
               space: "))
            if dim > 0:
                return dim
            else:
                print("Dimension must be positive.")
        except ValueError:
            print("Invalid input.")

def get_num_vectors():
    """
    Asks the user for the number of vectors.
    """
    while True:
        try:
            num = int(input("Enter the number of vectors: "))
            if num > 0:
                return num
            else:
                print("Number must be positive.")
        except ValueError:
            print("Invalid input.")

def get_vector(dim, name):
    """
    Inputs a vector of given dimension.
    """
    while True:
        elements = input(f"Enter {dim} elements for vector {name},
           separated by commas: ").split(',')
        if len(elements) != dim:
```

```
            print("Incorrect number of elements.")
            continue
        try:
            return np.array([float(x.strip()) for x in elements])
        except ValueError:
            print("Invalid input.")

def plot_vectors_2d(vectors, sum_vector):
    plt.figure()
    plt.grid()
    origin = np.zeros(2)

    for i, v in enumerate(vectors):
        plt.quiver(*origin, *v, scale=1, scale_units='xy',
           angles='xy', label=f'v{i+1}')

    plt.quiver(*origin, *sum_vector, scale=1, scale_units='xy',
       angles='xy', color='black', label='Sum')
    plt.legend()
    plt.xlabel("X-axis")
    plt.ylabel("Y-axis")
    plt.title("2D Vector Sum")
    plt.gca().set_aspect('equal')
    plt.savefig("vector_plot_2d.png")

def plot_vectors_3d(vectors, sum_vector):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    origin = np.zeros(3)

    for i, v in enumerate(vectors):
        ax.quiver(*origin, *v, label=f'v{i+1}')

    ax.quiver(*origin, *sum_vector, color='black', label='Sum')
    ax.set_xlabel("X-axis")
    ax.set_ylabel("Y-axis")
    ax.set_zlabel("Z-axis")
    ax.set_title("3D Vector Sum")
    ax.legend()
    plt.savefig("vector_plot_3d.png")

def main():
    dim = get_dimension()
    num_vectors = get_num_vectors()

    vectors = []
    for i in range(num_vectors):
```

```
        vectors.append(get_vector(dim, f"v{i+1}"))

    sum_vector = np.zeros(dim)

    for v in vectors:
        sum_vector += v

    for i, v in enumerate(vectors):
        print(f"Norm of v{i+1}: {np.linalg.norm(v):.4f}")

    print(f"Norm of sum: {np.linalg.norm(sum_vector):.4f}")

    if dim == 2:
        plot_vectors_2d(vectors, sum_vector)
    elif dim == 3:
        plot_vectors_3d(vectors, sum_vector)

if __name__ == "__main__":
    main()
```

# OUTPUT

Enter the dimension of the vector space (e.g., 2, 3, 4): 2
Enter how many vectors you want to input: 2 — Enter Vectors —
Enter the 2 elements for vector v1, separated by commas (e.g., '1,2'): -1,0.25
Enter the 2 elements for vector v2, separated by commas (e.g., '1,2'): 4,-0.125
— Individual Vector Norms —
Norm of vector v1 [-1.0000, 0.2500]: 1.0308
Norm of vector v2 [4.0000, -0.1250]: 4.0020
— Sum of Vectors —
The sum of the vectors is: [3.0000, 0.1250]
The norm of the sum of vectors is: 3.0026

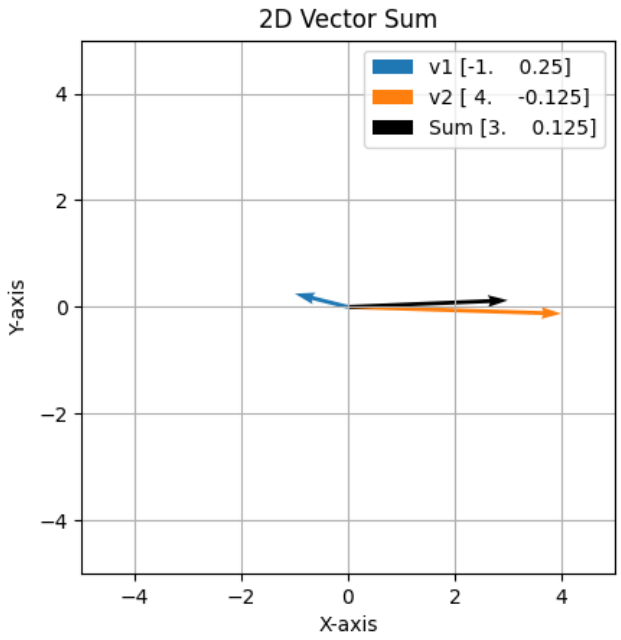



Figure 3: 2D vectors and their sums

## Distance between vectors

The distance between two vectors $u$ and $v$ in a vector space is defined as the norm of their difference:

$$\text{dist}(u, v) = \|u - v\|.$$

Here, $u - v$ represents the vector joining $v$ to $u$, and the norm $\|\cdot\|$ measures its magnitude. For the Euclidean norm in $\mathbb{R}^n$, this becomes

$$\|u - v\| = \sqrt{(u_1 - v_1)^2 + \cdots + (u_n - v_n)^2}.$$

This definition satisfies the properties of a distance: non-negativity, symmetry, and the triangle inequality.

Listing 4: Python code for finding the distance between 2 vectors

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Required for 3D plots

def get_dimension():
    """
    Asks the user for the dimension of the vector.
    Continues prompting until a valid positive integer is entered.
    """
    while True:
```

```
        try:
            dim_str = input("Enter the dimension of the vector: ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer.")

def get_vector(dim, vector_name="v"):
    """
    Gets a valid vector from the user for a given dimension.
    """
    while True:
        # Generate an example for the prompt based on the dimension
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements for {vector_name}
           vector , separated by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str_list = vector_str_input.split(',')

        if len(vector_str_list) != dim:
            print(f"Error: Incorrect number of elements. Please
               enter exactly {dim} elements.")
            continue

        try:
            # Convert string elements to floats and create a numpy
               array
            v = np.array([float(x.strip()) for x in
               vector_str_list])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numbers.")

def format_vector_for_display(vector):
    """
    Formats a numpy vector as a string with comma-separated
       elements.
    """
    return "(" + ", ".join([f"{x:.4f}" for x in vector]) + ")"

def calculate_vector_distance(vec1, vec2):
    """
    Calculates the Euclidean distance between two vectors.
```

```
    Args:
        vec1 (numpy.ndarray or list): The first vector.
        vec2 (numpy.ndarray or list): The second vector.

    Returns:
        float: The Euclidean distance between the two vectors.

    Raises:
        ValueError: If the dimensions of the vectors do not match.
    """
    # Convert inputs to numpy arrays to ensure consistent operations
    vec1 = np.array(vec1)
    vec2 = np.array(vec2)

    # Check if the dimensions (number of elements) of the vectors
       are the same
    if vec1.shape != vec2.shape:
        raise ValueError("Vectors must have the same dimension to
           calculate distance.")

    # Calculate the Euclidean distance
    # This is done by finding the difference between corresponding
       elements,
    # squaring each difference, summing them up, and then taking
       the square root.
    distance = np.linalg.norm(vec1 - vec2)

    return distance

def main_distance_calculator():
    """
    Main function to get two vectors from the user and calculate
       their Euclidean distance.
    """
    # Get the dimension from the user
    dim = get_dimension()

    # Get the first vector from the user
    vector1 = get_vector(dim, vector_name="first")
    print(f"\nFirst vector: {format_vector_for_display(vector1)}")

    # Get the second vector from the user
    vector2 = get_vector(dim, vector_name="second")
    print(f"Second vector: {format_vector_for_display(vector2)}")

    # Calculate the distance between the two vectors
```

```
    try:
        distance = calculate_vector_distance(vector1, vector2)
        print(f"\nEuclidean Distance between the two vectors:
            {distance:.4f}")

        # Plot the vectors if dimension is 2 or 3
        if dim == 2:
            plt.figure(figsize=(10, 10))
            ax = plt.gca()

            # Plot vector 1
            ax.quiver(0, 0, vector1[0], vector1[1], angles='xy',
                scale_units='xy', scale=1, color='blue',
                label=f'Vector 1:
                {format_vector_for_display(vector1)}', width=0.005,
                headwidth=5, headlength=8)
            # Plot vector 2
            ax.quiver(0, 0, vector2[0], vector2[1], angles='xy',
                scale_units='xy', scale=1, color='green',
                label=f'Vector 2:
                {format_vector_for_display(vector2)}', width=0.005,
                headwidth=5, headlength=8)

            # Plot line representing distance between vector heads
            ax.plot([vector1[0], vector2[0]], [vector1[1],
                vector2[1]], color='red', linestyle='--',
                label=f'Distance: {distance:.4f}', linewidth=2)

            # Calculate and plot the difference vector (vector2 -
                vector1)
            vector_diff = vector2 - vector1
            ax.quiver(0, 0, vector_diff[0], vector_diff[1],
                angles='xy', scale_units='xy', scale=1,
                color='purple', label=f'Difference (V2-V1):
                {format_vector_for_display(vector_diff)}',
                width=0.005, headwidth=5, headlength=8)

            # Set limits and labels
            max_abs = max(np.max(np.abs(vector1)),
                np.max(np.abs(vector2)),
                np.max(np.abs(vector_diff)), 1.5) # Ensure at least
                unit vectors are visible and includes difference
                vector
            ax.set_xlim([-max_abs - 0.5, max_abs + 0.5])
            ax.set_ylim([-max_abs - 0.5, max_abs + 0.5])
            ax.set_aspect('equal', adjustable='box')
            ax.axhline(0, color='gray', linewidth=0.5)
```

```
        ax.axvline(0, color='gray', linewidth=0.5)
        plt.xlabel('X-axis')
        plt.ylabel('Y-axis')
        plt.title(f'2D Vectors, Euclidean Distance, and
            Difference ({distance:.4f})')
        plt.legend()
        plt.grid(True)
        plt.show()
    elif dim == 3:
        fig = plt.figure(figsize=(12, 12))
        ax = fig.add_subplot(111, projection='3d')

        # Plot vector 1
        ax.quiver(0, 0, 0, vector1[0], vector1[1], vector1[2],
            color='blue', label=f'Vector 1:
            {format_vector_for_display(vector1)}',
            arrow_length_ratio=0.1)
        # Plot vector 2
        ax.quiver(0, 0, 0, vector2[0], vector2[1], vector2[2],
            color='green', label=f'Vector 2:
            {format_vector_for_display(vector2)}',
            arrow_length_ratio=0.1)

        # Plot line representing distance between vector heads
        ax.plot([vector1[0], vector2[0]], [vector1[1],
            vector2[1]], [vector1[2], vector2[2]], color='red',
            linestyle='--', label=f'Distance: {distance:.4f}',
            linewidth=2)

        # Calculate and plot the difference vector (vector2 -
            vector1)
        vector_diff = vector2 - vector1
        ax.quiver(0, 0, 0, vector_diff[0], vector_diff[1],
            vector_diff[2], color='purple', label=f'Difference
            (V2-V1): {format_vector_for_display(vector_diff)}',
            arrow_length_ratio=0.1)

        # Set limits and labels
        max_abs = max(np.max(np.abs(vector1)),
            np.max(np.abs(vector2)),
            np.max(np.abs(vector_diff)), 1.5) # Ensure at least
            unit vectors are visible and includes difference
            vector
        ax.set_xlim([-max_abs - 0.5, max_abs + 0.5])
        ax.set_ylim([-max_abs - 0.5, max_abs + 0.5])
        ax.set_zlim([-max_abs - 0.5, max_abs + 0.5])
        ax.set_xlabel('X-axis')
```

```
            ax.set_ylabel('Y-axis')
            ax.set_zlabel('Z-axis')
            plt.title(f'3D Vectors, Euclidean Distance, and
                Difference ({distance:.4f})')
            plt.legend()
            plt.grid(True)
            plt.show()
        else:
            print("\nPlotting is supported only for 2D and 3D
                vectors.")

    except ValueError as e:
        print(f"\nError: {e}")

if __name__ == "__main__":
    main_distance_calculator()
```

# OUTPUT

Enter the dimension of the vector: 3
Enter the 3 elements for first vector , separated by commas (e.g., '1,2,3'): 1,2,0

First vector: (1.0000, 2.0000, 0.0000)
Enter the 3 elements for second vector , separated by commas (e.g., '1,2,3'): -1,4,1
Second vector: (-1.0000, 4.0000, 1.0000)

Euclidean Distance between the two vectors: 3.0000

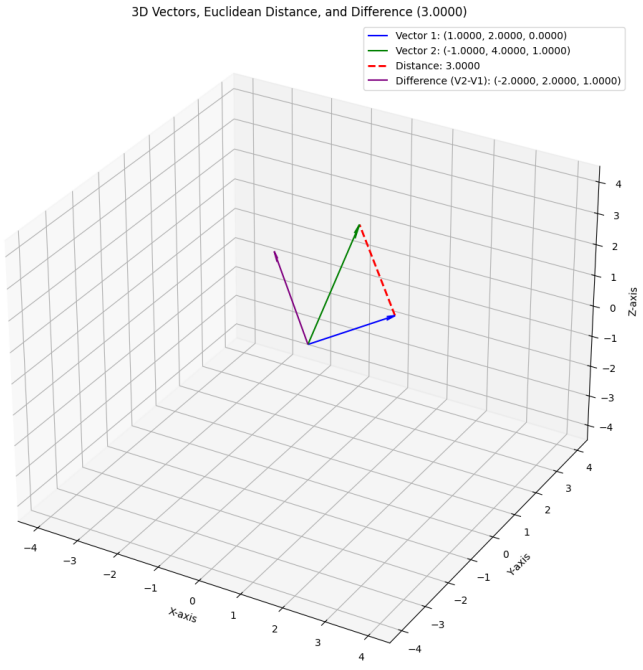



Figure 4: Distance between 2 3D vectors

## Explanation of the Program

The first program requests the dimension of the vector space and validates the input. It then accepts a vector with the specified number of components and computes its Euclidean norm using NumPy's built-in linear algebra functions. In the second program, the norm of vectors and norm of their sum is calculated and in the third, using the idea of norm the distance between 2 vectors is calculated. Robust exception handling ensures safe execution and meaningful error messages.

# 3 Unit Vectors

The following Python program allows the user to input a vector of arbitrary dimension and computes its unit vector in both the same and opposite directions.

### Code with Explanations

Listing 5: Python code to compute unit vectors

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Required for 3D plots
def get_dimension():
    """
```

```
    Asks the user for the dimension of the vector.
    Continues prompting until a valid positive integer is entered.
    """
    while True:
        try:
            dim_str = input("Enter the dimension of the vector: ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer.")
def get_vector(dim, vector_name="v"):
    """
    Gets a valid vector from the user for a given dimension.
    """
    while True:
        # Generate an example for the prompt based on the dimension
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements for vector
           {vector_name}, separated by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str_list = vector_str_input.split(',')
        if len(vector_str_list) != dim:
            print(f"Error: Incorrect number of elements. Please
               enter exactly {dim} elements.")
            continue
        try:
            # Convert string elements to floats and create a numpy
               array
            v = np.array([float(x.strip()) for x in
               vector_str_list])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numbers.")
def format_vector_for_display(vector):
    """
    Formats a numpy vector as a string with comma-separated
       elements.
    """
    return "(" + ", ".join([f"{x:.4f}" for x in vector]) + ")"
def main():
    """
    Main function to find unit vectors in the same and opposite
       directions.
```

```
    """
    # 1) Ask for the dimension
    dim = get_dimension()

    # 2) Get vector from user
    v = get_vector(dim)
    print(f"\nOriginal vector v: {format_vector_for_display(v)}")
    # Calculate the norm (magnitude) of the vector v
    norm_v = np.linalg.norm(v)
    if norm_v == 0:
        print("\nCannot calculate unit vectors for a zero vector.")
        return
    # 3) Compute unit vectors
    result1 = v / norm_v          # Unit vector in same direction
    result2 = -result1            # Unit vector in opposite
       direction
    # 4) Display the results
    print("\n--- Unit Vector Results ---")
    print(f"The unit vector of v {format_vector_for_display(v)}:")
    print(f"  In the direction of v:
       {format_vector_for_display(result1)}")
    print(f"  In the opposite direction of v :
       {format_vector_for_display(result2)}")
    # 5) Plot the vectors if dimension is 2 or 3
    if dim == 2:
        plt.figure(figsize=(8, 8))
        ax = plt.gca()
        # Plot original vector v
        ax.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy',
           scale=1, color='blue', label='Original Vector v',
           width=0.005, headwidth=5, headlength=8)
        # Plot unit vector in same direction
        ax.quiver(0, 0, result1[0], result1[1], angles='xy',
           scale_units='xy', scale=1, color='green', label='Unit
           Vector (same direction)', width=0.005, headwidth=5,
           headlength=8)
        # Plot unit vector in opposite direction
        ax.quiver(0, 0, result2[0], result2[1], angles='xy',
           scale_units='xy', scale=1, color='red', label='Unit
           Vector (opposite direction)', width=0.005, headwidth=5,
           headlength=8)
        # Set limits and labels
        max_val = max(abs(v).max(), 1.5) # Ensure at least unit
           vectors are visible
        ax.set_xlim([-max_val - 0.5, max_val + 0.5])
        ax.set_ylim([-max_val - 0.5, max_val + 0.5])
        ax.set_aspect('equal', adjustable='box')
```

```
        ax.axhline(0, color='gray', linewidth=0.5)
        ax.axvline(0, color='gray', linewidth=0.5)
        plt.xlabel('X-axis')
        plt.ylabel('Y-axis')
        plt.title('Vector and Unit Vectors in 2D')
        plt.legend()
        plt.grid(True)
        plt.show()
    elif dim == 3:
        fig = plt.figure(figsize=(10, 10))
        ax = fig.add_subplot(111, projection='3d')
        # Plot original vector v
        ax.quiver(0, 0, 0, v[0], v[1], v[2], color='blue',
            label='Original Vector v', arrow_length_ratio=0.1)
        # Plot unit vector in same direction
        ax.quiver(0, 0, 0, result1[0], result1[1], result1[2],
            color='green', label='Unit Vector (same direction)',
            arrow_length_ratio=0.1)
        # Plot unit vector in opposite direction
        ax.quiver(0, 0, 0, result2[0], result2[1], result2[2],
            color='red', label='Unit Vector (opposite direction)',
            arrow_length_ratio=0.1)
        # Set limits and labels
        max_val = max(abs(v).max(), 1.5) # Ensure at least unit
            vectors are visible
        ax.set_xlim([-max_val - 0.5, max_val + 0.5])
        ax.set_ylim([-max_val - 0.5, max_val + 0.5])
        ax.set_zlim([-max_val - 0.5, max_val + 0.5])
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        ax.set_zlabel('Z-axis')
        plt.title('Vector and Unit Vectors in 3D')
        plt.legend()
        plt.grid(True)
        plt.show()
    else:
        print("\nPlotting is supported only for 2D and 3D vectors.")
if __name__ == "__main__":
    main()
```

# OUTPUT

Enter the dimension of the vector: 2
Enter the 2 elements for vector v, separated by commas (e.g., '1,2'): -5,12
Original vector v: (-5.0000, 12.0000)
— Unit Vector Results —

The unit vector of v (-5.0000, 12.0000):
In the direction of v: (-0.3846, 0.9231)
In the opposite direction of v : (0.3846, -0.9231)

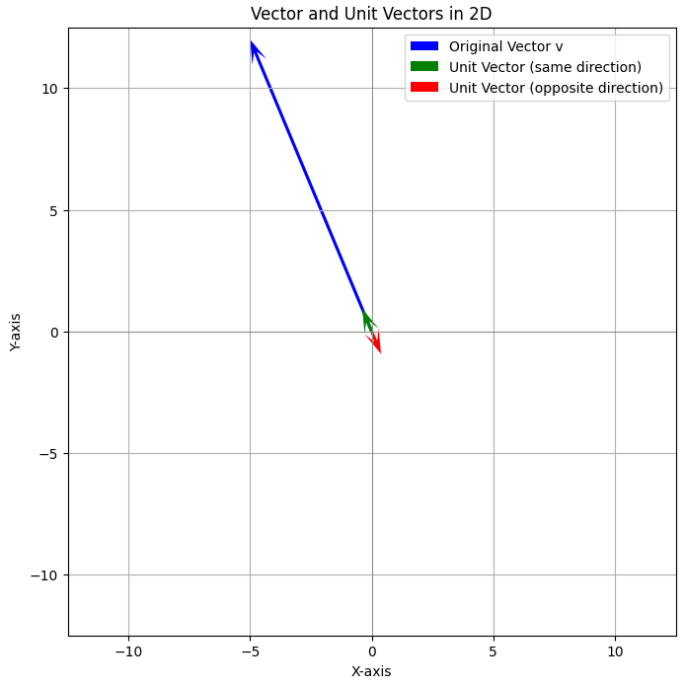



Figure 5: 2D vector and its unit vectors in same and opposite directions

**Explanation:** The `main` function orchestrates the program:

1. Prompts the user for the vector dimension.
2. Accepts the vector input from the user.
3. Computes the magnitude (norm) of the vector.
4. Calculates the unit vector in both the same and opposite directions.
5. Prints the results in a clear, formatted manner.

Listing 6: Python code

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # Required for 3D plots
def get_dimension():
    """
    Asks the user for the dimension of the vector.
    Continues prompting until a valid positive integer is entered.
    """
```

```
    while True:
        try:
            dim_str = input("Enter the dimension of the vector: ")
            dim = int(dim_str)
            if dim > 0:
                return dim
            else:
                print("Dimension must be a positive integer.")
        except ValueError:
            print("Invalid input. Please enter a single integer.")
def get_vector(dim, vector_name="v"):
    """
    Gets a valid vector from the user for a given dimension.
    """
    while True:
        # Generate an example for the prompt based on the dimension
        example = ','.join(str(i) for i in range(1, dim + 1))
        prompt = f"Enter the {dim} elements for vector
           {vector_name}, separated by commas (e.g., '{example}'): "
        vector_str_input = input(prompt)
        vector_str_list = vector_str_input.split(',')
        if len(vector_str_list) != dim:
            print(f"Error: Incorrect number of elements. Please
               enter exactly {dim} elements.")
            continue
        try:
            # Convert string elements to floats and create a numpy
               array
            v = np.array([float(x.strip()) for x in
               vector_str_list])
            return v
        except ValueError:
            print("Invalid input. Please ensure all elements are
               numbers.")
def format_vector_for_display(vector):
    """
    Formats a numpy vector as a string with comma-separated
       elements.
    """
    return "(" + ", ".join([f"{x:.4f}" for x in vector]) + ")"
def get_positive_float_input(prompt):
    """
    Asks the user for a positive float value.
    """
    while True:
        try:
            value_str = input(prompt)
```

```
            value = float(value_str)
            if value > 0:
                return value
            else:
                print("Value must be positive.")
        except ValueError:
            print("Invalid input. Please enter a valid number.")
def main():
    """
    Main function to find a vector of a given length in the
       direction of another vector.
    """
    # 1) Ask for the dimension
    dim = get_dimension()
     # 2) Get the reference vector from the user
    ref_vector = get_vector(dim, vector_name="reference")
    print(f"\nReference vector:
       {format_vector_for_display(ref_vector)}")
     # 3) Get the desired length from the user
    desired_length = get_positive_float_input("Enter the desired
       length for the new vector: ")
     # Calculate the norm (magnitude) of the reference vector
    norm_ref_vector = np.linalg.norm(ref_vector)
    if norm_ref_vector == 0:
        print("\nCannot determine direction from a zero reference
           vector.")
        return
    # 4) Calculate the unit vector of the reference vector
    unit_vector = ref_vector / norm_ref_vector
    # 5) Multiply the unit vector by the desired length
    result_vector = unit_vector * desired_length
    # 6) Display the results
    print("\n--- Result --- ")
    print(f"Desired length: {desired_length:.4f}")
    print(f"Vector of length {desired_length:.4f} in the direction
       of {format_vector_for_display(ref_vector)}:
       {format_vector_for_display(result_vector)}")
    # 7) Plot the vectors if dimension is 2 or 3
    if dim == 2:
        plt.figure(figsize=(8, 8))
        ax = plt.gca()
        # Plot reference vector
        ax.quiver(0, 0, ref_vector[0], ref_vector[1], angles='xy',
           scale_units='xy', scale=1, color='blue',
           label='Reference Vector', width=0.005, headwidth=5,
           headlength=8)
        # Plot result vector
```

```
        ax.quiver(0, 0, result_vector[0], result_vector[1],
            angles='xy', scale_units='xy', scale=1, color='green',
            label=f'Vector of length {desired_length:.2f}',
            width=0.005, headwidth=5, headlength=8)
        # Set limits and labels
        max_abs_ref = np.max(np.abs(ref_vector))
        max_abs_result = np.max(np.abs(result_vector))
        max_val = max(max_abs_ref, max_abs_result, 1.5) # Ensure at
            least unit vectors are visible
        ax.set_xlim([-max_val - 0.5, max_val + 0.5])
        ax.set_ylim([-max_val - 0.5, max_val + 0.5])
        ax.set_aspect('equal', adjustable='box')
        ax.axhline(0, color='gray', linewidth=0.5)
        ax.axvline(0, color='gray', linewidth=0.5)
        plt.xlabel('X-axis')
        plt.ylabel('Y-axis')
        plt.title(f'Vector and New Vector of Length
            {desired_length:.2f} in 2D')
        plt.legend()
        plt.grid(True)
        plt.show()
    elif dim == 3:
        fig = plt.figure(figsize=(10, 10))
        ax = fig.add_subplot(111, projection='3d')
        # Plot reference vector
        ax.quiver(0, 0, 0, ref_vector[0], ref_vector[1],
            ref_vector[2], color='blue', label='Reference Vector',
            arrow_length_ratio=0.1)
        # Plot result vector
        ax.quiver(0, 0, 0, result_vector[0], result_vector[1],
            result_vector[2], color='green', label=f'Vector of
            length {desired_length:.2f}', arrow_length_ratio=0.1)
        # Set limits and labels
        max_abs_ref = np.max(np.abs(ref_vector))
        max_abs_result = np.max(np.abs(result_vector))
        max_val = max(max_abs_ref, max_abs_result, 1.5) # Ensure at
            least unit vectors are visible
        ax.set_xlim([-max_val - 0.5, max_val + 0.5])
        ax.set_ylim([-max_val - 0.5, max_val + 0.5])
        ax.set_zlim([-max_val - 0.5, max_val + 0.5])
        ax.set_xlabel('X-axis')
        ax.set_ylabel('Y-axis')
        ax.set_zlabel('Z-axis')
        plt.title(f'Vector and New Vector of Length
            {desired_length:.2f} in 3D')
        plt.legend()
        plt.grid(True)
```

```
        plt.show()
    else:
        print("\nPlotting is supported only for 2D and 3D vectors.")
if __name__ == "__main__":
    main()
```

# OUTPUT

```
Enter the dimension of the vector: 2
Enter the 2 elements for vector reference, separated by commas (e.g., '1,2'): 1,1
Reference vector: (1.0000, 1.0000)
Enter the desired length for the new vector: 4
— Result —
Desired length: 4.0000
Vector of length 4.0000 in the direction of (1.0000, 1.0000): (2.8284, 2.8284)
```

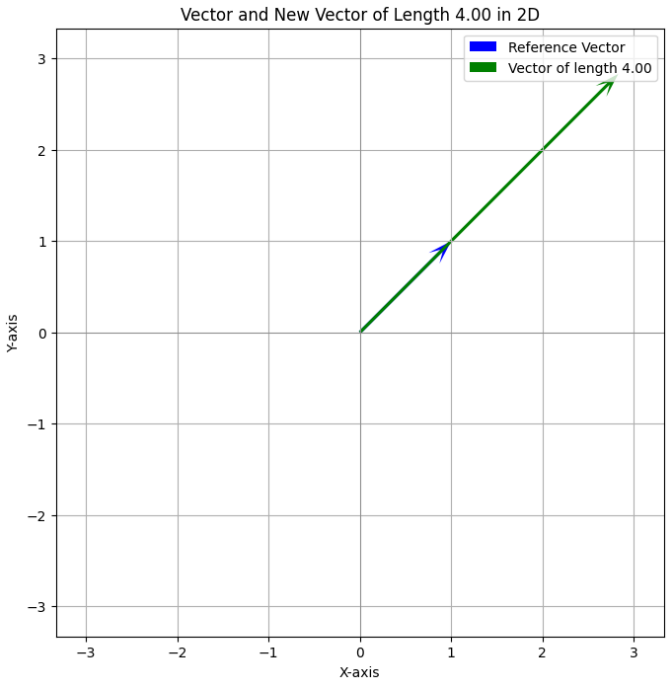



Figure 6: 2D representation

# 4 Dot Product (Scalar Product)

Let

$$\mathbf{a} = (a_1, a_2, \ldots, a_n) \text{ and } \mathbf{b} = (b_1, b_2, \ldots, b_n) \text{ be two vectors in } \mathbb{R}^n.$$

## Definition

The **dot product** (or scalar product) of $\mathbf{a}$ and $\mathbf{b}$ is defined as

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \sum_{i=1}^{n} a_i b_i.$$

## Geometric Interpretation

If $\theta$ is the angle between $\mathbf{a}$ and $\mathbf{b}$, then

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta,$$

where $\|\mathbf{a}\|$ and $\|\mathbf{b}\|$ are the magnitudes (norms) of the vectors.

## Properties

1. Commutative: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$
2. Distributive over addition: $\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}$
3. Scalar multiplication: $(k\mathbf{a}) \cdot \mathbf{b} = k(\mathbf{a} \cdot \mathbf{b})$ for any scalar $k$
4. Positive-definite: $\mathbf{a} \cdot \mathbf{a} = \|\mathbf{a}\|^2 \geq 0$, and equality holds if and only if $\mathbf{a} = \mathbf{0}$

## Applications

- Finding the angle between two vectors: $\cos\theta = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$
- Checking orthogonality: $\mathbf{a} \perp \mathbf{b} \iff \mathbf{a} \cdot \mathbf{b} = 0$
- Projection of $\mathbf{a}$ on $\mathbf{b}$: $\text{proj}_{\mathbf{b}}\mathbf{a} = \dfrac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{b}\|^2}\mathbf{b}$

1)Given $\mathbf{u} = (3, 4)$ and $\mathbf{v} = (2, -3)$, find:

1. $\mathbf{u} \cdot \mathbf{v}$

$$\mathbf{u} \cdot \mathbf{v} = 3 \cdot 2 + 4 \cdot (-3) = 6 - 12 = -6$$

2. $\mathbf{u} \cdot \mathbf{u}$

$$\mathbf{u} \cdot \mathbf{u} = 3^2 + 4^2 = 9 + 16 = 25$$

3. $\|\mathbf{u}\|^2$

$$\|\mathbf{u}\|^2 = \mathbf{u} \cdot \mathbf{u} = 25$$

4. $(\mathbf{u} \cdot \mathbf{v})\mathbf{v}$

$$(\mathbf{u} \cdot \mathbf{v})\mathbf{v} = (-6)(2, -3) = (-12, 18)$$

5. $\mathbf{u} \cdot (5\mathbf{v})$

$$\mathbf{u} \cdot (5\mathbf{v}) = 5(\mathbf{u} \cdot \mathbf{v}) = 5(-6) = -30$$

2)Given $\mathbf{u} = (-1, 1, -2)$ and $\mathbf{v} = (1, -3, -2)$, find:

1. $\mathbf{u} \cdot \mathbf{v}$

$$\mathbf{u} \cdot \mathbf{v} = (-1)(1) + (1)(-3) + (-2)(-2) = -1 - 3 + 4 = 0$$

2. $\mathbf{u} \cdot \mathbf{u}$

$$\mathbf{u} \cdot \mathbf{u} = (-1)^2 + 1^2 + (-2)^2 = 1 + 1 + 4 = 6$$

3. $\|\mathbf{u}\|^2$

$$\|\mathbf{u}\|^2 = \mathbf{u} \cdot \mathbf{u} = 6$$

4. $(\mathbf{u} \cdot \mathbf{v})\mathbf{v}$

$$(\mathbf{u} \cdot \mathbf{v})\mathbf{v} = 0 \cdot (1, -3, -2) = (0, 0, 0)$$

5. $\mathbf{u} \cdot (5\mathbf{v})$

$$\mathbf{u} \cdot (5\mathbf{v}) = 5(\mathbf{u} \cdot \mathbf{v}) = 5 \cdot 0 = 0$$

3)Given $\mathbf{u} = (3, 1)$ and $\mathbf{v} = (-2, 4)$, find the angle $\theta$ between them.

$$\text{Recall: } \cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}.$$

**Step 1: Compute the dot product**

$$\mathbf{u} \cdot \mathbf{v} = 3 \cdot (-2) + 1 \cdot 4 = -6 + 4 = -2$$

**Step 2: Compute the magnitudes**

$$\|\mathbf{u}\| = \sqrt{3^2 + 1^2} = \sqrt{9 + 1} = \sqrt{10}, \quad \|\mathbf{v}\| = \sqrt{(-2)^2 + 4^2} = \sqrt{4 + 16} = \sqrt{20}$$

**Step 3: Compute** $\cos\theta$

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \frac{-2}{\sqrt{10} \cdot \sqrt{20}} = \frac{-2}{\sqrt{200}} = \frac{-2}{10\sqrt{2}} = -\frac{1}{5\sqrt{2}}$$

**Step 4: Find** $\theta$

$$\theta = \cos^{-1}\left(-\frac{1}{5\sqrt{2}}\right)$$

4)Given $\mathbf{u} = (1, 1, 1)$ and $\mathbf{v} = (2, 1, -1)$, find the angle $\theta$ between them.

**Step 1: Compute the dot product**

$$\mathbf{u} \cdot \mathbf{v} = 1 \cdot 2 + 1 \cdot 1 + 1 \cdot (-1) = 2 + 1 - 1 = 2$$

**Step 2: Compute the magnitudes**

$$\|\mathbf{u}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3}, \quad \|\mathbf{v}\| = \sqrt{2^2 + 1^2 + (-1)^2} = \sqrt{4 + 1 + 1} = \sqrt{6}$$

**Step 3: Compute** $\cos\theta$

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \frac{2}{\sqrt{3} \cdot \sqrt{6}} = \frac{2}{\sqrt{18}} = \frac{2}{3\sqrt{2}} = \frac{\sqrt{2}}{3}$$

**Step 4: Find** $\theta$

$$\theta = \cos^{-1}\left(\frac{\sqrt{2}}{3}\right)$$

5)Vectors orthogonal to $\mathbf{u} = (0, 5)$

Let $\mathbf{v} = (x, y)$ be orthogonal to $\mathbf{u}$.

$$\mathbf{u} \cdot \mathbf{v} = 0 \implies 0 \cdot x + 5 \cdot y = 0 \implies y = 0$$

Hence all vectors orthogonal to $\mathbf{u}$ are of the form $\mathbf{v} = (x, 0), \quad x \in \mathbb{R}$.

6)Vectors orthogonal to $\mathbf{u} = (4, -1, 0)$

Let $\mathbf{v} = (x, y, z)$ be orthogonal to $\mathbf{u}$.

$$\mathbf{u} \cdot \mathbf{v} = 0 \implies 4x + (-1)y + 0 \cdot z = 0 \implies 4x - y = 0 \implies y = 4x$$

Hence all vectors orthogonal to $\mathbf{u}$ are of the form $\mathbf{v} = (x, 4x, z), \quad x, z \in \mathbb{R}$.

7)Given $\mathbf{u} = (2, 18)$, $\mathbf{v} = \left(\frac{3}{2}, -\frac{1}{6}\right)$, determine whether they are orthogonal, parallel, or neither.

**Step 1: Check orthogonality**

$$\mathbf{u} \cdot \mathbf{v} = 2 \cdot \frac{3}{2} + 18 \cdot \left(-\frac{1}{6}\right) = 3 - 3 = 0$$

Since $\mathbf{u} \cdot \mathbf{v} = 0$, the vectors are orthogonal.

8)Given $\mathbf{u} = (0, 1, 0)$, $\mathbf{v} = (1, -2, 0)$, determine whether they are orthogonal, parallel, or neither.

**Step 1: Check orthogonality**

$$\mathbf{u} \cdot \mathbf{v} = 0 \cdot 1 + 1 \cdot (-2) + 0 \cdot 0 = -2$$

Since $\mathbf{u} \cdot \mathbf{v} \neq 0$, the vectors are not orthogonal.

**Step 2: Check parallelism**

$$\mathbf{u} = (0, 1, 0), \quad \mathbf{v} = (1, -2, 0)$$

$$\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} = \frac{0 \cdot 1 + 1 \cdot (-2) + 0 \cdot 0}{\sqrt{0^2 + 1^2 + 0^2}\sqrt{1^2 + (-2)^2 + 0^2}} = \frac{-2}{1 \cdot \sqrt{5}} = -\frac{2}{\sqrt{5}}$$

$$\text{Since } |\cos\theta| = \frac{2}{\sqrt{5}} \neq 1, \text{ the vectors are not parallel.}$$

The vectors are neither orthogonal nor parallel.

9)Explain what is known about $\theta$, the angle between $\mathbf{u}$ and $\mathbf{v}$, if:

1. $\mathbf{u} \cdot \mathbf{v} = 0$
2. $\mathbf{u} \cdot \mathbf{v} > 0$
3. $\mathbf{u} \cdot \mathbf{v} < 0$

**Solution:**

The angle $\theta$ between two vectors $\mathbf{u}$ and $\mathbf{v}$ is related to the dot product by

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta$$

1. **If $\mathbf{u} \cdot \mathbf{v} = 0$:**

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta = 0$$

Since $\|\mathbf{u}\| \neq 0$ and $\|\mathbf{v}\| \neq 0$, it must be that

$$\cos\theta = 0 \implies \theta = 90^\circ$$

Hence, the vectors are orthogonal (perpendicular to each other).

2. **If $\mathbf{u} \cdot \mathbf{v} > 0$:**

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta > 0$$

Therefore,

$$\cos\theta > 0 \implies 0^\circ < \theta < 90^\circ$$

Hence, the angle between the vectors is acute.

3. **If $\mathbf{u} \cdot \mathbf{v} < 0$:**

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta < 0$$

Therefore,

$$\cos\theta < 0 \implies 90^\circ < \theta < 180^\circ$$

Hence, the angle between the vectors is obtuse.

# Inner Product

## Inner Product (Real Vector Spaces)

Let $V$ be a vector space over $\mathbb{R}$. An *inner product* on $V$ is a function

$$\langle \cdot, \cdot \rangle : V \times V \to \mathbb{R}$$

satisfying the following axioms for all $u, v, w \in V$ and $\alpha \in \mathbb{R}$:

1. **Symmetry:**
$$\langle u, v \rangle = \langle v, u \rangle$$

2. **Additivity (Linearity in the first argument):**
$$\langle u + w, v \rangle = \langle u, v \rangle + \langle w, v \rangle$$

3. **Homogeneity (Scalar multiplication in the first argument):**
$$\langle \alpha u, v \rangle = \alpha \langle u, v \rangle$$

4. **Positive Definiteness:**
$$\langle v, v \rangle \geq 0 \quad \text{and} \quad \langle v, v \rangle = 0 \iff v = 0$$

- The norm (or length) of a vector $v$ is defined using the inner product as
$$\|v\| = \sqrt{\langle v, v \rangle}.$$

  **Properties:**
  - $\|v\| \geq 0$ and $\|v\| = 0 \iff v = 0$ (positive definiteness)
  - $\|\alpha v\| = |\alpha| \, \|v\|$ for any scalar $\alpha$

- The distance between two vectors $u$ and $v$ is defined as
$$d(u, v) = \|u - v\| = \sqrt{\langle u - v, u - v \rangle}.$$
  This satisfies all the usual properties of a distance function.

- If $u, v \neq 0$, the angle $\theta$ between $u$ and $v$ is defined by
$$\cos \theta = \frac{\langle u, v \rangle}{\|u\| \, \|v\|}, \quad 0 \leq \theta \leq \pi.$$

**Remarks:**

- $\langle u, v \rangle > 0 \implies$ acute angle
- $\langle u, v \rangle = 0 \implies$ right angle
- $\langle u, v \rangle < 0 \implies$ obtuse angle

## Note: Complex Vector Spaces

If $V$ is a vector space over $\mathbb{C}$, the inner product is usually defined to satisfy:

1. **Conjugate Symmetry:**
$$\langle u, v\rangle = \overline{\langle v, u\rangle}$$

2. **Linearity and Homogeneity in the first argument:**
$$\langle \alpha u + w, v\rangle = \alpha\langle u, v\rangle + \langle w, v\rangle$$

3. **Positive Definiteness:**
$$\langle v, v\rangle \geq 0, \quad \langle v, v\rangle = 0 \iff v = 0$$

**Remark:** In complex spaces, the inner product is only linear in the first argument; in the second argument it is conjugate-linear.

Let $V$ be a real inner product space (for complex spaces, see notes below), and let $u, v \in V$. If $V$ is a complex inner product space:

- The norm and distance formulas are the same:
$$\|v\| = \sqrt{\langle v, v\rangle}, \quad d(u, v) = \|u - v\|$$

- The inner product $\langle u, v\rangle$ may be complex. Hence, the cosine of the angle is often defined using the real part:
$$\cos\theta = \frac{\mathrm{Re}(\langle u, v\rangle)}{\|u\|\,\|v\|}.$$

- This ensures that $\theta$ is a real number and lies between 0 and $\pi$.

## Projection of a Vector

Let $u, v \in V$ with $v \neq 0$.

**Real Inner Product Spaces:** The projection of $u$ onto $v$ is defined as

$$\mathrm{proj}_v(u) = \frac{\langle u, v\rangle}{\langle v, v\rangle}\, v$$

where $\langle \cdot, \cdot \rangle$ is the inner product in $V$. The vector $u - \mathrm{proj}_v(u)$ is orthogonal to $v$, i.e.,

$$\langle u - \mathrm{proj}_v(u), v\rangle = 0.$$

**Complex Inner Product Spaces (Note):** The formula is formally the same:

$$\text{proj}_v(u) = \frac{\langle u, v\rangle}{\langle v, v\rangle}\, v$$

but now $\langle u, v\rangle$ may be complex. The vector $u - \text{proj}_v(u)$ is orthogonal to $v$ with respect to the complex inner product:

$$\langle u - \text{proj}_v(u), v\rangle = 0.$$

**Remark:** In complex spaces, the scalar $\frac{\langle u,v\rangle}{\langle v,v\rangle}$ can be complex, but the geometric idea of projection remains the same as in real spaces.

In inner product spaces :
**Cauchy–Schwarz Inequality:**

$$|\langle u, v\rangle| \le \|u\|\,\|v\|, \quad u, v \in V$$

**Pythagorean Theorem:**

$$u \perp v \implies \|u + v\|^2 = \|u\|^2 + \|v\|^2$$

**Triangle Inequality**

$$\|u + v\| \le \|u\| + \|v\|, \quad u, v \in V$$

1)Verify that $\langle u, v\rangle = u_1v_1 + 2u_2v_2$ is an inner product on $\mathbb{R}^2$.

Let $u = (u_1, u_2),\ v = (v_1, v_2) \in \mathbb{R}^2$. Define

$$\langle u, v\rangle = u_1v_1 + 2u_2v_2.$$

Symmetry:

$$\langle u, v\rangle = u_1v_1 + 2u_2v_2 = v_1u_1 + 2v_2u_2 = \langle v, u\rangle$$

Additivity:

Let $w = (w_1, w_2) \in \mathbb{R}^2$

$$\langle u + w, v\rangle = (u_1 + w_1)v_1 + 2(u_2 + w_2)v_2 = (u_1v_1 + 2u_2v_2) + (w_1v_1 + 2w_2v_2) = \langle u, v\rangle + \langle w, v\rangle$$

Homogeneity:

For $\alpha \in \mathbb{R}$,

$$\langle \alpha u, v\rangle = (\alpha u_1)v_1 + 2(\alpha u_2)v_2 = \alpha(u_1v_1 + 2u_2v_2) = \alpha\langle u, v\rangle$$

Positive Definiteness:

$$\langle u, u\rangle = u_1^2 + 2u_2^2 \geq 0$$

and

$$\langle u, u\rangle = 0 \implies u_1^2 + 2u_2^2 = 0 \implies u_1 = 0, u_2 = 0 \implies u = 0 \text{(and vice-versa)}$$

All four axioms are satisfied, hence

$$\langle u, v\rangle = u_1 v_1 + 2u_2 v_2$$

is an inner product on $\mathbb{R}^2$.

2)For weights $c_i > 0$ on $\mathbb{R}^n$, prove that

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i\, u_i v_i$$

is an inner product.

Let $u = (u_1, u_2, \ldots, u_n)$ and $v = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n$, and let $c_i > 0$ for $i = 1, 2, \ldots, n$. Define

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i\, u_i v_i.$$

Symmetry:

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i\, u_i v_i = \sum_{i=1}^{n} c_i\, v_i u_i = \langle v, u\rangle$$

Additivity:
Let $w = (w_1, \ldots, w_n) \in \mathbb{R}^n$.

$$\langle u + w, v\rangle = \sum_{i=1}^{n} c_i(u_i + w_i)v_i = \sum_{i=1}^{n} c_i u_i v_i + \sum_{i=1}^{n} c_i w_i v_i = \langle u, v\rangle + \langle w, v\rangle$$

Homogeneity:
For $\alpha \in \mathbb{R}$,

$$\langle \alpha u, v\rangle = \sum_{i=1}^{n} c_i(\alpha u_i)v_i = \alpha \sum_{i=1}^{n} c_i u_i v_i = \alpha\langle u, v\rangle$$

Positive Definiteness:

$$\langle u, u\rangle = \sum_{i=1}^{n} c_i u_i^2 \geq 0$$

and

$$\langle u, u\rangle = 0 \implies \sum_{i=1}^{n} c_i u_i^2 = 0 \implies u_i = 0 \text{ for all } i \implies u = 0 \text{(and vice-versa)}$$

All four axioms are satisfied, hence

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i u_i v_i$$

is an inner product on $\mathbb{R}^n$ for any weights $c_i > 0$.

3)Give a counterexample showing that the formula

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i\, u_i v_i$$

fails to define an inner product if one weight $c_i = 0$ or $c_i < 0$.

**Case 1: One weight is 0:** Let $n = 2$, $c_1 = 1$, $c_2 = 0$, and consider $u = (0, 1) \in \mathbb{R}^2$.

$$\langle u, u\rangle = c_1 u_1^2 + c_2 u_2^2 = 1 \cdot 0^2 + 0 \cdot 1^2 = 0$$

but $u \neq 0$.
$\implies$ Positive definiteness fails when a weight is 0.

**Case 2: One weight is negative:** Let $c_1 = 1$, $c_2 = -1$, and consider $u = (0, 1) \in \mathbb{R}^2$.

$$\langle u, u\rangle = c_1 u_1^2 + c_2 u_2^2 = 1 \cdot 0^2 + (-1) \cdot 1^2 = -1 < 0$$

$\implies$ Positive definiteness fails when a weight is negative.
The formula

$$\langle u, v\rangle = \sum_{i=1}^{n} c_i\, u_i v_i$$

defines an inner product only if all weights $c_i > 0$. If any weight is zero or negative, the positive definiteness axiom is violated.

4)On $C[0, 1]$, compute $\|x\|$, $\|x^2\|$, $\langle x, x^2\rangle$, and the angle between them, where the inner product is defined as

$$\langle f, g\rangle = \int_0^1 f(t)g(t)\, dt.$$

Compute $\|x\|$:

$$\|x\| = \sqrt{\langle x, x\rangle} = \sqrt{\int_0^1 x^2\, dx} = \sqrt{\frac{1}{3}} = \frac{1}{\sqrt{3}}$$

Compute $\|x^2\|$:

$$\|x^2\| = \sqrt{\langle x^2, x^2\rangle} = \sqrt{\int_0^1 x^4\, dx} = \sqrt{\frac{1}{5}} = \frac{1}{\sqrt{5}}$$

Compute $\langle x, x^2 \rangle$:

$$\langle x, x^2 \rangle = \int_0^1 x \cdot x^2 \, dx = \int_0^1 x^3 \, dx = \frac{1}{4}$$

Compute the angle between $x$ and $x^2$:

$$\cos\theta = \frac{\langle x, x^2 \rangle}{\|x\| \, \|x^2\|} = \frac{1/4}{(1/\sqrt{3})(1/\sqrt{5})} = \frac{\sqrt{15}}{4}$$

$$\theta = \cos^{-1}\left(\frac{\sqrt{15}}{4}\right)$$

5)On $M_{2,2}$, compute $\|A\|$ and check whether the two given matrices

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

are orthogonal with respect to the inner product defined for any $A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$, $B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$ by

$$\langle A, B \rangle = a_{11}b_{11} + a_{21}b_{21} + a_{12}b_{12} + a_{22}b_{22}.$$

Norm of $A$

$$\|A\| = \sqrt{\langle A, A \rangle} = \sqrt{1 \cdot 1 + 3 \cdot 3 + 2 \cdot 2 + 4 \cdot 4} = \sqrt{1 + 9 + 4 + 16} = \sqrt{30}$$

Check orthogonality with $B$

$$\langle A, B \rangle = 1 \cdot 0 + 3 \cdot (-1) + 2 \cdot 1 + 4 \cdot 0 = 0 - 3 + 2 + 0 = -1$$

Since $\langle A, B \rangle \neq 0$, the matrices $A$ and $B$ are **not orthogonal**.
Let the columns of $B$ be:

$$\begin{bmatrix} 3 & 2 \\ -1 & -1 \end{bmatrix}$$

Check orthogonality:

$$\langle A, B \rangle = \mathbf{u}_1 \cdot \mathbf{v}_1 + \mathbf{u}_2 \cdot \mathbf{v}_2 = (1 \cdot 3 + 3 \cdot (-1)) + (2 \cdot 2 + 4 \cdot (-1)) = (3 - 3) + (4 - 4) = 0$$

$$B = \begin{bmatrix} 3 & 2 \\ -1 & -1 \end{bmatrix}$$

is orthogonal to A.

# 5 Cauchy–Schwarz Inequality

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ be two vectors. The **Cauchy–Schwarz inequality** states that

$$|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\| \, \|\mathbf{v}\|.$$

Here, $\mathbf{u} \cdot \mathbf{v}$ denotes the dot product of the vectors, and

$$\|\mathbf{u}\| = \sqrt{\mathbf{u} \cdot \mathbf{u}}$$

is the magnitude (or norm) of $\mathbf{u}$.

**Geometric interpretation:** Since

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta,$$

where $\theta$ is the angle between the vectors, the inequality follows from the fact that

$$|\cos\theta| \leq 1.$$

Equality holds if and only if $\mathbf{u}$ and $\mathbf{v}$ are linearly dependent.

## 5.1 Verification of the Cauchy–Schwarz Inequality

Let $\mathbf{u} = (3, 4)$ and $\mathbf{v} = (2, -3)$.

First, compute the dot product:

$$\mathbf{u} \cdot \mathbf{v} = (3)(2) + (4)(-3) = 6 - 12 = -6.$$

Thus,

$$|\mathbf{u} \cdot \mathbf{v}| = 6.$$

Next, compute the magnitudes of the vectors:

$$\|\mathbf{u}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5,$$

$$\|\mathbf{v}\| = \sqrt{2^2 + (-3)^2} = \sqrt{13}.$$

Now, compute the product of the magnitudes:

$$\|\mathbf{u}\| \, \|\mathbf{v}\| = 5\sqrt{13}.$$

Since

$$6 \leq 5\sqrt{13},$$

the Cauchy–Schwarz inequality

$$|\mathbf{u} \cdot \mathbf{v}| \leq \|\mathbf{u}\| \, \|\mathbf{v}\|$$

is verified for the given vectors.

# 6 Triangle Inequality

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$. The **triangle inequality** for vectors states that

$$\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|.$$

**Explanation:** This inequality expresses the fact that the length of one side of a triangle is always less than or equal to the sum of the lengths of the other two sides. When vectors are placed head-to-tail, $\mathbf{u} + \mathbf{v}$ represents the third side of the triangle.

Equality occurs if and only if $\mathbf{u}$ and $\mathbf{v}$ point in the same direction.

## 6.1 Verification of the Triangle Inequality

Let $\mathbf{u} = (1, 1, 1)$ and $\mathbf{v} = (0, 1, -2)$.

First, compute the sum of the vectors:

$$\mathbf{u} + \mathbf{v} = (1 + 0,\ 1 + 1,\ 1 - 2) = (1, 2, -1).$$

Next, compute the magnitudes of the vectors:

$$\|\mathbf{u}\| = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3},$$

$$\|\mathbf{v}\| = \sqrt{0^2 + 1^2 + (-2)^2} = \sqrt{5}.$$

Now, compute the magnitude of the sum:

$$\|\mathbf{u} + \mathbf{v}\| = \sqrt{1^2 + 2^2 + (-1)^2} = \sqrt{6}.$$

Finally, compute the sum of the magnitudes:

$$\|\mathbf{u}\| + \|\mathbf{v}\| = \sqrt{3} + \sqrt{5}.$$

Since

$$\sqrt{6} \leq \sqrt{3} + \sqrt{5},$$

the triangle inequality

$$\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$$

is verified for the given vectors.

# 7 Pythagorean Theorem

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ be vectors such that

$$\mathbf{u} \cdot \mathbf{v} = 0.$$

This means that $\mathbf{u}$ and $\mathbf{v}$ are orthogonal (perpendicular).

The **Pythagorean theorem for vectors** states that

$$\|\mathbf{u} + \mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2.$$

**Proof:**

$$\|\mathbf{u}+\mathbf{v}\|^2 = (\mathbf{u}+\mathbf{v})\cdot(\mathbf{u}+\mathbf{v}) = \mathbf{u}\cdot\mathbf{u} + 2\,\mathbf{u}\cdot\mathbf{v} + \mathbf{v}\cdot\mathbf{v}.$$

Since $\mathbf{u}\cdot\mathbf{v} = 0$, this simplifies to

$$\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2.$$

This result generalizes the classical Pythagorean theorem to vector spaces.

## 7.1 Verification of the Pythagorean Theorem

Let $\mathbf{u} = (3, 4, -2)$ and $\mathbf{v} = (4, -3, 0)$.

First, compute the dot product:

$$\mathbf{u}\cdot\mathbf{v} = (3)(4) + (4)(-3) + (-2)(0) = 12 - 12 + 0 = 0.$$

Since $\mathbf{u}\cdot\mathbf{v} = 0$, the vectors are orthogonal.

Next, compute the magnitudes of the vectors:

$$\|\mathbf{u}\|^2 = 3^2 + 4^2 + (-2)^2 = 9 + 16 + 4 = 29,$$

$$\|\mathbf{v}\|^2 = 4^2 + (-3)^2 + 0^2 = 16 + 9 + 0 = 25.$$

Now, compute the sum of the vectors:

$$\mathbf{u}+\mathbf{v} = (7, 1, -2).$$

Then, compute the square of the magnitude of the sum:

$$\|\mathbf{u}+\mathbf{v}\|^2 = 7^2 + 1^2 + (-2)^2 = 49 + 1 + 4 = 54.$$

Since

$$\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 = 29 + 25 = 54,$$

the Pythagorean theorem

$$\|\mathbf{u}+\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2$$

is verified for the given vectors.

# 8 Verification of the Statement

1)The given statement is

$$\|\mathbf{v}\| = |v_1 + v_2 + \cdots + v_n|.$$

This statement is **false**.

**Counterexample:** Let $\mathbf{v} = (1, -1, 0, \ldots, 0) \in \mathbb{R}^n$.

Then,

$$\|\mathbf{v}\| = \sqrt{1^2 + (-1)^2} = \sqrt{2},$$

while

$$|v_1 + v_2 + \cdots + v_n| = |1 - 1 + 0 + \cdots + 0| = 0.$$

Since

$$\sqrt{2} \neq 0,$$

the given statement is false.
2)The given statement is:

$$\text{If } \mathbf{u} \cdot \mathbf{v} < 0, \text{ then the angle between } \mathbf{u} \text{ and } \mathbf{v} \text{ is acute.}$$

This statement is **false**.
**Reason / Counterexample:** For any two nonzero vectors $\mathbf{u}$ and $\mathbf{v}$,

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \, \|\mathbf{v}\| \cos\theta,$$

where $\theta$ is the angle between the vectors.
If $\mathbf{u} \cdot \mathbf{v} < 0$, then

$$\cos\theta < 0,$$

which implies that

$$\theta > 90^\circ.$$

Hence, the angle between $\mathbf{u}$ and $\mathbf{v}$ is **obtuse**, not acute.
Therefore, the given statement is false.

# 9 Meaningless expression

1) Explain why the expression $\|u \cdot v\|$ is meaningless.

The expression $\|u \cdot v\|$ is meaningless because $u \cdot v$ denotes the dot product of vectors $u$ and $v$, which is a scalar, whereas the norm $\|\cdot\|$ is defined for vectors, not for scalars.

2) Explain why the expression $u + (u \cdot v)$ is meaningless.

The expression $u + (u \cdot v)$ is meaningless because $u$ is a vector while $u \cdot v$ is a scalar, and vector addition between a vector and a scalar is not defined.

# 10 Wronskian and Linear Independence

The Wronskian of a set of $n$ functions $f_1, f_2, \ldots, f_n$, which are $(n-1)$ times differentiable on an interval $I$, is defined as the determinant

$$W(f_1, f_2, \ldots, f_n)(x) = \begin{vmatrix} f_1(x) & f_2(x) & \cdots & f_n(x) \\ f_1'(x) & f_2'(x) & \cdots & f_n'(x) \\ f_1''(x) & f_2''(x) & \cdots & f_n''(x) \\ \vdots & \vdots & \ddots & \vdots \\ f_1^{(n-1)}(x) & f_2^{(n-1)}(x) & \cdots & f_n^{(n-1)}(x) \end{vmatrix}.$$

The Wronskian is used to study the linear independence of functions. If there exists a point $x_0 \in I$ such that $W(x_0) \neq 0$, then the functions $f_1, f_2, \ldots, f_n$ are linearly independent on the interval $I$. If $W(x) = 0$ for all $x \in I$, and the functions are solutions to a homogeneous linear differential equation, they are linearly dependent.

Listing 7: Wronskian computation using SymPy

```
import sympy
import numpy as np
import matplotlib.pyplot as plt

def wronskian(functions, x):
    """
    Calculates the Wronskian matrix and its determinant for a list
       of functions
    with respect to a variable x.

    Args:
        functions (list): A list of sympy symbolic functions.
        x (sympy.Symbol): The symbolic variable with respect to
           which to differentiate.

    Returns:
        tuple: A tuple containing the Wronskian matrix
           (sympy.Matrix) and
               its determinant (sympy.Expr).
    """
    n = len(functions)
    if n == 0:
        return sympy.Matrix([]), 0

    # Create the Wronskian matrix
    # Each row corresponds to a derivative order, each column to a
       function.
    wronskian_matrix = sympy.Matrix.zeros(n, n)
    # Loop for derivative order (rows)
    for j in range(n):
        # Loop for functions (columns)
        for i in range(n):
            current_func = functions[i]
            # Calculate the j-th derivative of the i-th function
            derivative = sympy.diff(current_func, x, j)
            wronskian_matrix[j, i] = derivative

    # Calculate the determinant of the Wronskian matrix
    determinant = wronskian_matrix.det()
    return wronskian_matrix, determinant

# --- Interactive Example Usage ---
```

```
x = sympy.symbols('x')

print("Enter functions as symbolic expressions (e.g., 'sin(x)',
    'x**2', 'exp(x)').")

input_functions_str = []

while True:
    try:
        num_functions = int(input("How many functions do you want
            to enter? "))
        if num_functions < 0:
            print("Please enter a non-negative number.")
        else:
            break
    except ValueError:
        print("Invalid input. Please enter an integer.")

for i in range(num_functions):
    func_str = input(f"Enter function {i + 1}: ")
    input_functions_str.append(func_str)

if not input_functions_str:
    print("No functions entered. Exiting.")
else:
    try:
        # Convert string inputs to sympy symbolic functions,
            ensuring 'e' is recognized as sympy.E
        symbolic_functions = [sympy.sympify(f_str, locals={'x': x,
            'sin': sympy.sin, 'cos': sympy.cos, 'exp': sympy.exp,
            'log': sympy.log, 'e': sympy.E}) for f_str in
            input_functions_str]

        print(f"\nCalculating Wronskian for functions:
            {symbolic_functions} with respect to {x}")
        wronskian_mat, wronskian_det =
            wronskian(symbolic_functions, x)

        print("\n--- Wronskian Matrix ---")
        display(wronskian_mat) # Using display for better matrix
            rendering in Colab

        # Print the simplified determinant
        print("\n--- Wronskian Determinant ---")
        simplified_det = sympy.simplify(wronskian_det)
        print(simplified_det)
```

```
# --- Plotting the Wronskian Determinant and individual
    functions ---
print("\n--- Plotting Wronskian Determinant and Input
    Functions ---")

# Convert the sympy expression to a numerical function
wronskian_func = sympy.lambdify(x, simplified_det, 'numpy')

# Define the range for x
x_vals = np.linspace(-5, 5, 400) # From -5 to 5 with 400
    points

# Evaluate the Wronskian function over the x range
y_vals_wronskian = wronskian_func(x_vals)

# Handle scalar output for constant Wronskian determinant
if np.isscalar(y_vals_wronskian):
    y_vals_wronskian = np.full_like(x_vals,
        y_vals_wronskian)

# Create the plot
plt.figure(figsize=(12, 8))
plt.plot(x_vals, y_vals_wronskian, label=f'Wronskian:
    {simplified_det}', linewidth=2, linestyle='--')

# Plot each individual function
for func_sym in symbolic_functions:
    func_lambdified = sympy.lambdify(x, func_sym, 'numpy')
    func_y_vals = func_lambdified(x_vals)

    # Handle scalar output for constant functions
    if np.isscalar(func_y_vals):
        func_y_vals = np.full_like(x_vals, func_y_vals)

    plt.plot(x_vals, func_y_vals, label=f'Function:
        {func_sym}')

plt.title('Plot of Wronskian Determinant and Input
    Functions')
plt.xlabel('x')
plt.ylabel('Value')
plt.grid(True)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.legend()
plt.show()
```

```
        # --- Analysis of Linear Independence ---
        print("\n--- Analysis of Linear Independence ---")
        # Check if the determinant is identically zero. sympy.Eq
            can compare symbolic expressions.
        # If simplified_det is a constant (like 0), sympy.Eq will
            handle it.
        if sympy.Eq(simplified_det, 0):
            print("The Wronskian determinant is identically zero
                for all x. This test is inconclusive; the functions
                may or may not be linearly independent.")
        else:
            # If the Wronskian is not identically zero, it means
                it's non-zero at at least one point.
            print("The Wronskian determinant is not identically
                zero. Therefore, the functions are linearly
                independent.")

    except (sympy.SympifyError, NameError) as e:
        print(f"Error parsing function: {e}. Please ensure correct
            symbolic syntax.")
```

# Problems

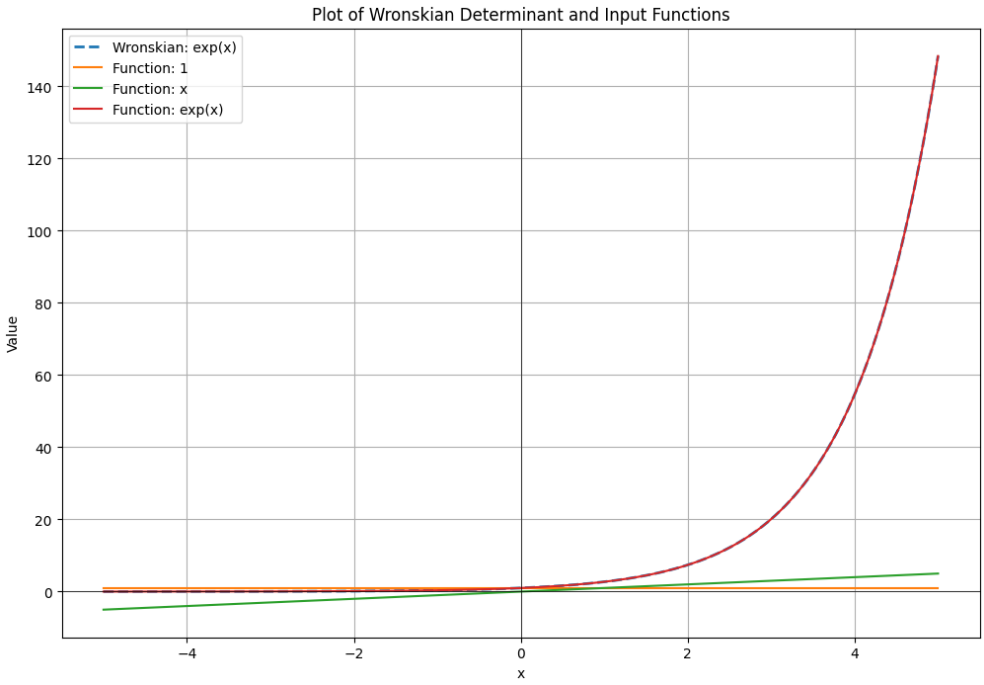
1)Find the Wronskian of the set of functions

$\{1, x, e^x\}$.

OUTPUT =>Enter functions as symbolic expressions (e.g., 'sin(x)', 'x**2', 'exp(x)').
How many functions do you want to enter? 3
Enter function 1:1
Enter function 2: x
Enter function 3: e**x
Calculating Wronskian for functions: [1, x, exp(x)] with respect to x
— Wronskian Matrix —

$$\begin{bmatrix} 1 & x & e^x \\ 0 & 1 & e^x \\ 0 & 0 & e^x \end{bmatrix}$$

— Wronskian Determinant — exp(x)
— Plotting Wronskian Determinant and Input Functions —




— Analysis of Linear Independence —
The Wronskian determinant is not identically zero. Therefore, the functions are **linearly independent**.

2)Find the Wronskian of the set of functions

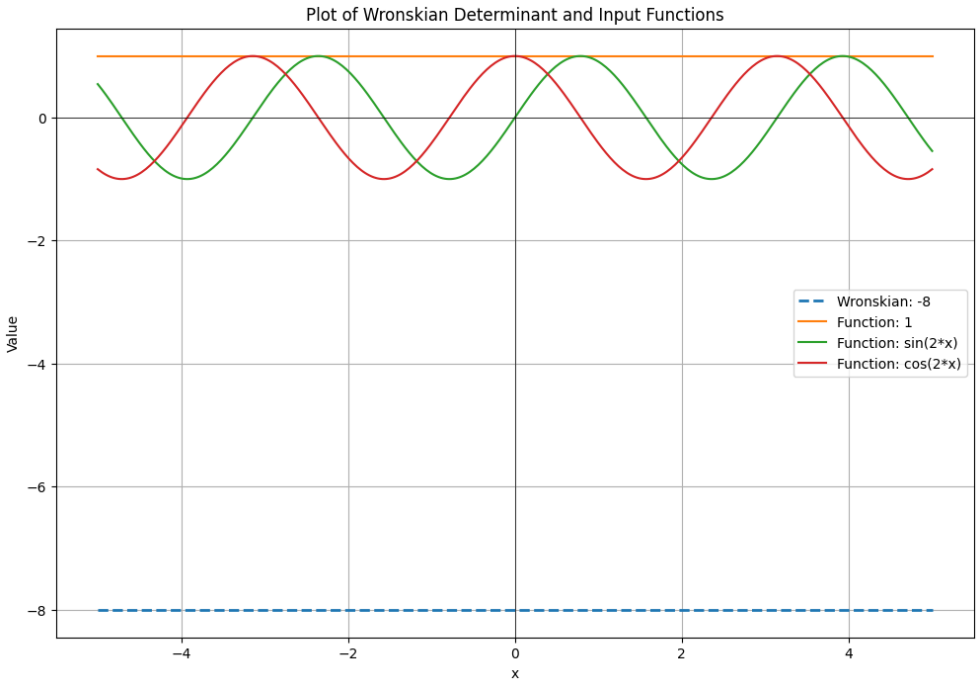
{1, sin(2x), cos(2x)}.

OUTPUT =>Enter functions as symbolic expressions (e.g., 'sin(x)', 'x**2', 'exp(x)').
How many functions do you want to enter? 3
Enter function 1:1
Enter function 2: sin(2*x)
Enter function 3: cos(2*x)
Calculating Wronskian for functions: [1, sin(2*x), cos(2*x)] with respect to x
— Wronskian Matrix —

$$\begin{bmatrix} 1 & sin(2x) & cos(2x) \\ 0 & 2cos(2x) & -2sin(2x) \\ 0 & -4sin(2x) & -4cos(2x) \end{bmatrix}$$

— Wronskian Determinant —
-8

—Plotting Wronskian Determinant and Input Functions —




— Analysis of Linear Independence —
The Wronskian determinant is not identically zero. Therefore, the functions are **linearly independent**.

3)Test the following set of solutions for linear independence:

$$y'' + 6y' + 9y = 0, \qquad \{e^{-3x}, xe^{-3x}\}$$

OUTPUT =>Enter functions as symbolic expressions (e.g., 'sin(x)', 'x**2', 'exp(x)').
How many functions do you want to enter? 2
Enter function 1: e**(-3*x)
Enter function 2: x*e**(-3*x)

Calculating Wronskian for functions: [exp(-3*x), x*exp(-3*x)] with respect to x
— Wronskian Matrix —

$$\begin{bmatrix} e^{-3x} & xe^{-3x} \\ -3e^{-3x} & -3xe^{-3x} + e^{-3x} \end{bmatrix}$$

— Wronskian Determinant —
exp(-6*x)
— Analysis of Linear Independence — The Wronskian determinant is not identically zero. Therefore, the functions are **linearly independent**.

4)Test the following set of solutions for linear independence:

$$y''' - 6y'' + 11y' - 6y = 0, \qquad \{e^x, e^{2x}, e^x - e^{2x}\}.$$

OUTPUT =>Enter functions as symbolic expressions (e.g., 'sin(x)', 'x**2', 'exp(x)').
How many functions do you want to enter? 3
Enter function 1: e**x
Enter function 2: e**(2*x)
Enter function 3: e**x-e**(2*x)

Calculating Wronskian for functions: [exp(x), exp(2*x), -exp(2*x) + exp(x)] with respect to x

— Wronskian Matrix —

$$\begin{bmatrix} e^x & e^{2x} & -e^{2x}+e^x \\ e^x & 2e^{2x} & -2e^{2x}+e^x \\ e^x & 4e^{2x} & (1-4e^x)e^x \end{bmatrix}$$

— Wronskian Determinant —
0

If the Wronskian of a set of functions is zero, linear dependence does not follow in general. However, for solutions of a linear homogeneous differential equation with continuous coefficients, an identically zero Wronskian implies linear dependence.Since these functions are given to be solutions of the differential equation (which can also be verified directly), the given set of solutions is linearly dependent.

$$y_1 = e^x, \quad y_2 = e^{2x}, \quad y_3 = e^x - e^{2x}$$

$$y''' - 6y'' + 11y' - 6y = 0$$

—

1) Check $y_1 = e^x$:

$$\begin{aligned} y_1' &= e^x \\ y_1'' &= e^x \\ y_1''' &= e^x \\ y_1''' - 6y_1'' + 11y_1' - 6y_1 &= e^x - 6e^x + 11e^x - 6e^x \\ &= (1 - 6 + 11 - 6)e^x = 0 \end{aligned}$$

2) Check $y_2 = e^{2x}$:

$$\begin{aligned} y_2' &= 2e^{2x} \\ y_2'' &= 4e^{2x} \\ y_2''' &= 8e^{2x} \\ y_2''' - 6y_2'' + 11y_2' - 6y_2 &= 8e^{2x} - 6(4e^{2x}) + 11(2e^{2x}) - 6e^{2x} \\ &= (8 - 24 + 22 - 6)e^{2x} = 0 \end{aligned}$$

3) Check $y_3 = e^x - e^{2x}$:

$$
\begin{aligned}
y_3' &= e^x - 2e^{2x} \\
y_3'' &= e^x - 4e^{2x} \\
y_3''' &= e^x - 8e^{2x} \\
y_3''' - 6y_3'' + 11y_3' - 6y_3 &= (e^x - 8e^{2x}) - 6(e^x - 4e^{2x}) + 11(e^x - 2e^{2x}) - 6(e^x - e^{2x}) \\
&= (1 - 6 + 11 - 6)e^x + (-8 + 24 - 22 + 6)e^{2x} = 0
\end{aligned}
$$

Hence, all three functions satisfy the differential equation.
Since the Wronskian is identically zero and all functions satisfy the same linear homogeneous differential equation, the set

$$\{e^x,\, e^{2x},\, e^x - e^{2x}\}$$

is **linearly dependent**.

# Conclusion

This assignment has explored the fundamental concepts of matrix theory and vector spaces. By integrating Python programming and visualizations using `matplotlib`, the assignment bridged theoretical results with computational practice, allowing abstract ideas to be visualized and verified effectively.

Overall, these concepts form a strong mathematical foundation for subsequent topics in matrix theory and linear algebra. The combination of rigorous theory, problem-solving, and computational visualization highlights the versatility and applicability of linear algebra in both pure and applied mathematical contexts.